SquadHQ

Architektur & Prototyp

P3 | Software-Projekt WS 2026/27 | Hochschule Flensburg | 28.10.2026

Trainingszusagen, Strafenkatalog und Teamkasse an einem Ort, statt WhatsApp und Zettelwirtschaft.

Heute

1. Fortschritt & Kundenfeedback
2. Architektur & Entwurf
3. Traceability
4. Live-Demo & Tests
5. Risiken & offene Fragen

Abbas Sharba
Requirements

Kevin Wolfram
Architektur & Entwurf

Jegor Geer
Implementierung

Ömer-Faruk Simsek
QA/Testing

Kirill Korsun
Management

Sprint 1: geplant vs. erreicht

Geplant (aus P2)

- Prisma-Schema und Seeds mit TSVK-Strafenkatalog
- Trainer-Anwesenheitsliste mit 1-Klick-Bestätigung
- Spieler-Ansicht: eigenes Kassenkonto

Erreicht

- Alle drei Ziele umgesetzt, Prototyp läuft (PR #31)
- Zusätzlich: Strafvorschlag mit Freigabe, Verwerfen, Storno und Audit-Log
- 5 Diagramme, Begründung, Traceability
- 12 automatisierte Tests, alle bestanden

Verschoben auf Sprint 2

- Kassenwart-Ansicht, Zahlungen verbuchen
- Trainings anlegen, Fehlzeiten-Übersicht
- Echter Login (Supabase Auth)

Projekt-Tracking live

Board: github.com/users/Kirillv2/projects/1
Commits: Repo → Insights → Contributors

live zeigen

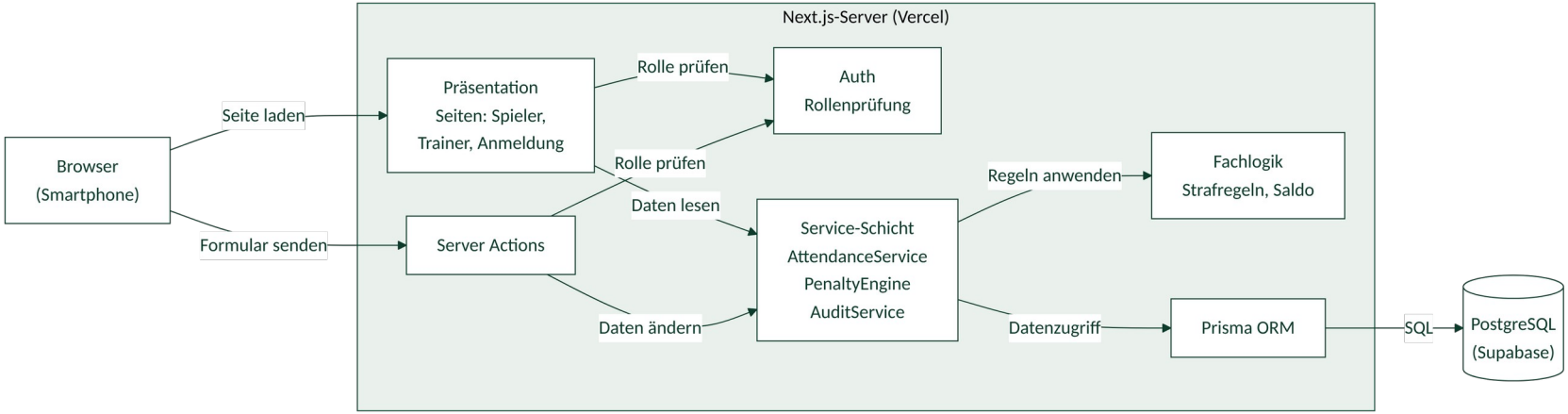
Kundenfeedback: von P2 in den Prototyp

Quelle	Feedback aus P2	Umgesetzt im Prototyp	Stand
Trainer	*Fehltermin erst bestätigen, bevor eine Strafe eingetragen wird*	Status PENDING_APPROVAL, Knöpfe Freigeben / Verwerfen	**fertig**
Kassenwart	*Nachvollziehen, wer eine Strafe eingetragen, geändert oder storniert hat*	AuditLog bei jeder Statusänderung, nur anhängen	**fertig**
Kassenwart	*Bar, PayPal, Teilzahlung; Status offen / teilweise / bezahlt*	Zahlungsstatus im Datenmodell; Verbuchen folgt	**teilweise**
Spieler	*Absagen fällt schwer, ein Klick mit optionalem Grund*	Dabei / Fehle entschuldigt / Noch unsicher, Grund optional	**fertig**

Nächster Schritt: Feedback am Prototyp

Wer: Trainer, Kassenwart und Spieler des TSV Kronshagen · Wie: Demo am Smartphone, kurze Aufgaben · Bis: P4 (11.11.) · Ergebnis: in die Feedback-Matrix

Komponenten und Konnektoren



1 Konnektoren

HTTPS GET/POST vom Smartphone, Funktionsaufrufe im Server, SQL zur Datenbank

2 Lesen und Schreiben getrennt

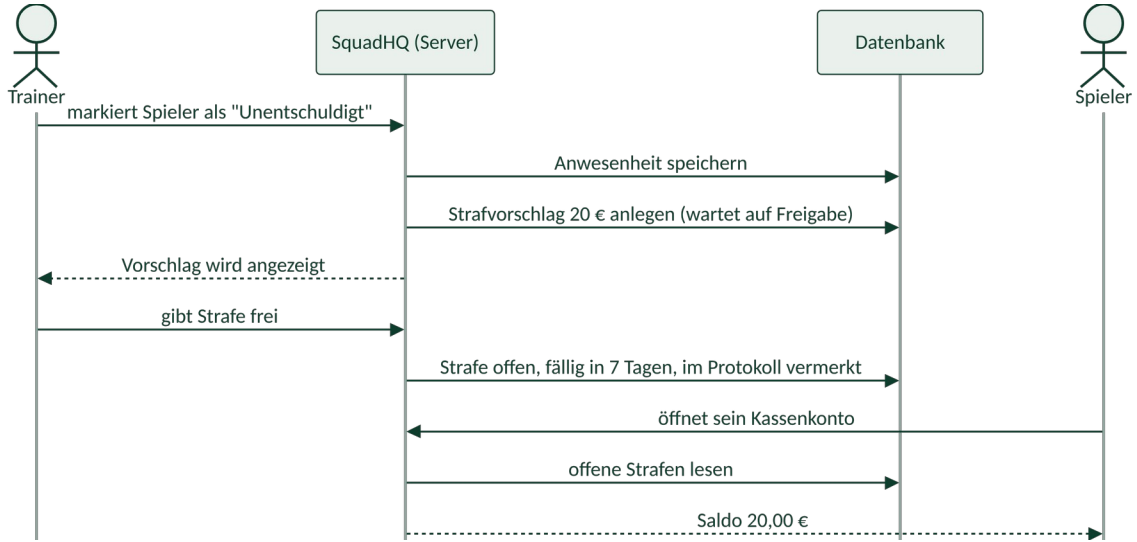
Seiten lesen über Services, geschrieben wird nur über Server Actions

3 Rollen zentral

Jede Seite und Action prüft die Rolle in Auth (REQ-SEC-01)

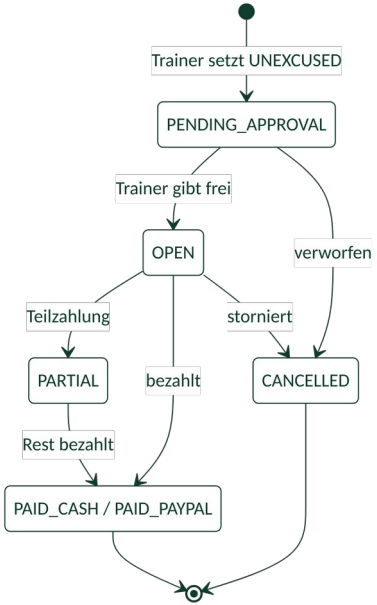
Verhalten: Strafvorschlag mit Freigabe

Sequenzdiagramm REQ-PEN-02



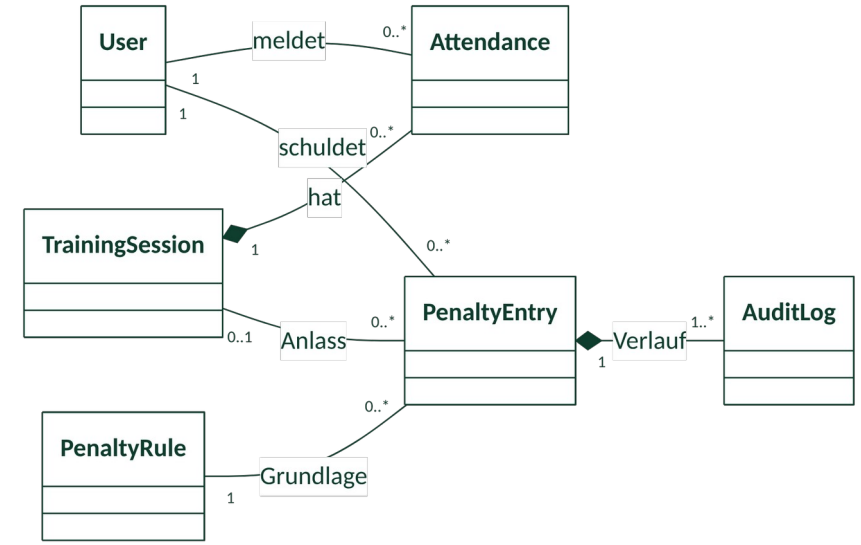
Kernidee: Die App schlägt nur vor, der Trainer entscheidet. Jeder Statuswechsel landet im Audit-Log.

Zustände einer Strafe

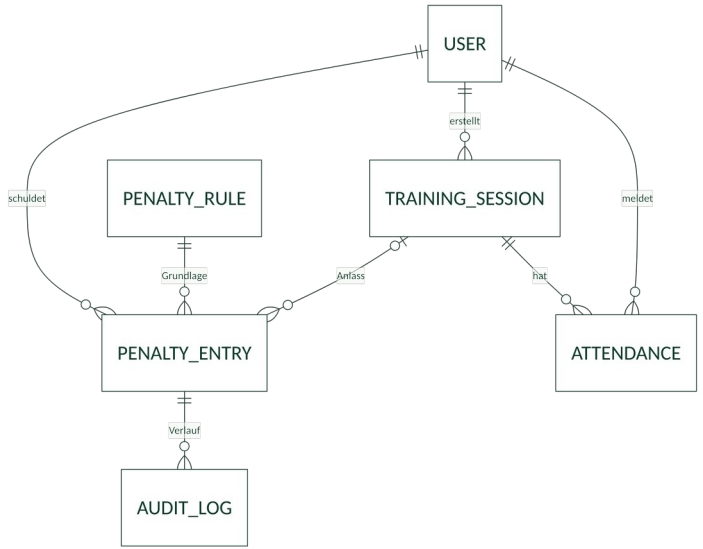


Klassendiagramm und Datenmodell

Klassendiagramm (Fachklassen)



ER-Modell (= prisma/schema.prisma)



Beträge in Cent: keine Rundungsfehler

PENDING_APPROVAL: Trainer bestätigt erst

CANCELLED statt Löschen: Storno nachvollziehbar

Architekturstil: Schichtenarchitektur

Client-Server + Schichten

Präsentation
Seiten (React Server Components)

Anwendung
Server Actions + Auth

Services
Attendance, Penalty, Audit

Fachlogik
reine Funktionen, unit-getestet

Datenzugriff
Prisma → PostgreSQL

Warum diese Architektur?

1. **Mobil am Spielfeldrand:** Web-App ohne Installation, Server-Rendering hält Seiten klein
2. **Nachvollziehbare Kasse:** Alle Schreibzugriffe über Services, jede Statusänderung im Audit-Log
3. **Testbarkeit:** Fachregeln ohne DB und UI, direkt unit-testbar
4. **Wenig Betriebsaufwand:** Ein Repo, ein Deployment (Vercel), verwaltete DB (Supabase)

Verworfen

Native App: App-Store-Hürde, zweite Codebasis
Eigenes REST-Backend: mehr Deployments ohne Mehrwert
Microservices: für 5 Personen und 6 Entitäten zu aufwendig
Supabase direkt im Browser: Regeln und Audit-Log manipulierbar

Traceability: vom Feedback bis zum Test

1 Feedback
Trainer: „erst bestätigen können“

2 Anforderung
REQ-PEN-02 Strafvorschlag mit Bestätigung

3 Entwurf
Sequenzdiagramm + Zustandsautomat

4 Code
penaltyEngine.ts, auditService.ts

5 Test
AK1–AK3 in penaltyFlow.db.test.ts

REQ	Anforderung	Komponente	Stand P3
REQ-SEC-01	Rollen Trainer, Kassenwart, Spieler	Auth, Server Actions	teilweise (Demo-Login)
REQ-ATT-01	1-Klick-Zu-/Absage	Spieler-Ansicht, AttendanceService	teilweise (Trainings anlegen fehlt)
REQ-ATT-02	Trainer-Dashboard Anwesenheit	Trainer-Dashboard, AttendanceService	teilweise (Fehlzeiten fehlt)
REQ-PEN-01	TSVK-Strafenkatalog	PenaltyRule (Datenmodell, Seed)	im Modell
REQ-PEN-02	Strafvorschlag mit Bestätigung	PenaltyEngine, AuditService	**fertig**
REQ-PEN-03	Kasse mit Zahlungsstatus und Saldo	Spieler-Kassenkonto, balance()	teilweise (Kassenwart fehlt)

Prototyp: Kernablauf live

1 **Spieler sagt ab**

Ein Klick, Grund optional

2 **Trainer bestätigt**

Noah „Unentschuldigt“ setzen

3 **Vorschlag freigeben**

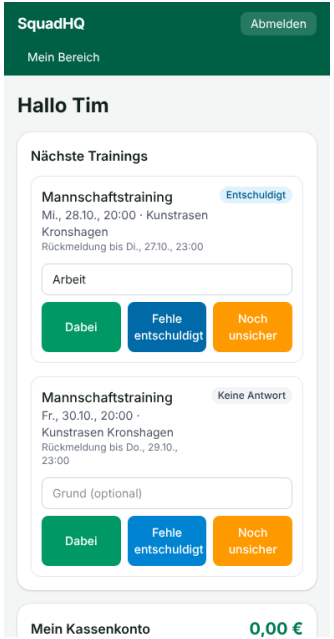
TSVK Pos. 8, 20,00 €

4 **Spieler sieht Saldo**

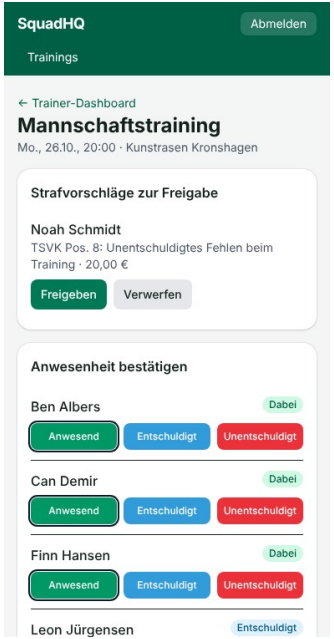
20,00 € offen, fällig in 7 Tagen

live zeigen

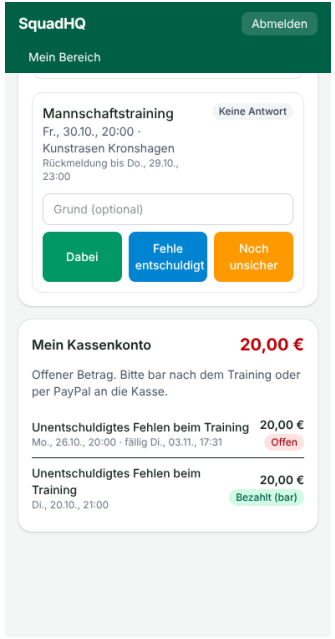
Ablauf: docs/demo-ablauf.md



Spieler-Ansicht



Trainer: Freigabe



Kassenkonto

Qualitätssicherung: 12 Tests, alle grün

12/12

automatisierte Tests bestanden (Vitest)

9	Unit-Tests Fachlogik
3	Integrationstests mit DB

Akzeptanzkriterien REQ-PEN-02 als Tests

1	**When** Trainer setzt UNEXCUSED **Then** genau ein Entwurf über 20,00 € (PENDING_APPROVAL)	grün
2	**When** Trainer klickt „Freigeben“ **Then** Status OPEN, Fälligkeit in 7 Tagen	grün
3	**When** nachträglich EXCUSED **Then** Strafe storniert, im Audit-Log protokolliert	grün

Review

Alle Diagramme mit Schema und Code abgeglichen, Abweichungen per PR korrigiert (#41–#43)

Bis P4

- E2E-Tests (Playwright)
- CI mit GitHub Actions
- Coding-Standards festlegen

Risiken & nächste Schritte

Risiken und Maßnahmen

1. **Commits ungleich verteilt**
   eigene Tickets und Commits für alle 5
2. **Sonderstrafen bremsen MVP**
   Getränkekisten bleiben in Phase 2
3. **Nur Demo-Login**
   Supabase Auth in Sprint 2

Sprint 2 bis P4 (11.11.)

1. **Kassenwart-Ansicht**
   Zahlungen bar, PayPal, Teilzahlung
2. **Trainings anlegen**
   plus Fehlzeiten-Übersicht
3. **Feedback-Runde**
   Prototyp beim TSV Kronshagen testen